ZW MCP — Architecture & the CLM API surface

One MCP server exposing every Docusign product API to Claude Desktop, claude.ai, Claude Code and custom UIs.

Woodward Systems · demo account
84 tools · 12 product APIs
Verified against live APIs 16 Sep 2026

1 · SHAPE OF THE SYSTEM

```
MCP client (Claude Desktop / claude.ai / your app)
   │  streamable HTTP + bearer token
   ▼
ZW MCP  :8787   Node + TypeScript
   │  JWT Grant → access token (cached, auto-refresh)
   ├─► eSignature   demo.docusign.net/restapi
   ├─► Navigator ─┐
   ├─► Maestro    ├ api-d.docusign.com (IAM)
   ├─► Workspaces ┘
   ├─► CLM         *discovered per account*
   └─► Rooms · Click · Admin · Web Forms · Notary …
```

One always-on service. Clients hold **only** a URL and a bearer token — no integration key, no JWT, no base URIs ever reach the client.

2 · AUTHENTICATION (ONE CHAIN, ALL PRODUCTS)

- **JWT Grant with impersonation.** An RSA-signed assertion is exchanged for an access token acting *as* a named user. No interactive login, so it survives reboots unattended.
- **Consent is once per integration key.** The impersonated user must accept the scope set a single time.
- **Scopes are the trap.** One scope the account lacks fails the *entire* consent grant, and Docusign records a consent that silently omits it — so "I clicked Accept" is not proof. A per-scope prober isolates the offender.

CLM needs `spring_read` + `spring_write` (`content` for upload/download).

3 · TWO-TIER TOOL DESIGN

Docusign has 1,000+ endpoints; one MCP tool each would drown any client's context window. So:

- **Curated tools** — hand-written for what demos actually do, with trimmed responses and model-facing descriptions (`clm_search_documents`, `esign_create_envelope`).
- **One `<product>_raw_request` per API** — reaches every remaining endpoint with auth, base URI and `{accountId}` handled. Coverage is complete from day one; curation is an ergonomics layer, not a limit.

4 · WHY CLM NEEDED ITS OWN CLIENT

Eleven products fit one shared HTTP client. CLM (formerly SpringCM) does not, for two reasons:

- **Hosts are data-centre specific and not knowable up front.** You discover them per account at runtime, and there are **four different hosts** — object, task, upload, download.
- **Paths carry the account id after the version:** `https://{host}/{version}/{accountId}/{resource}`

```
GET https://authuat.springcm.com/api/v2/{accountId}/account
  → ApiBaseUrl         apiuatna11.springcm.com
    ApiBaseUploadUrl   apiuploaduatna11.springcm.com
    ApiBaseDownloadUrl apidownloaduatna11.springcm.com
    version            v2      (cached for process life)
```

The raw escape hatch routes through this same resolver — otherwise `clm_raw_request` would silently target the wrong URL.

5 · CLM ENDPOINTS — VERIFIED LIVE

Path (after `/{version}/{accountId}`)	Status	Returns here
`/folders`	OK	79 root folders
`/workflows`	OK	87 definitions
`/attributegroups`	OK	20 groups
`/doclauncherconfigurations`	OK	Doc Launcher configs
`/documentsearchtasks`	OK	async full-text search
`/groups`	OK	security groups
`/documents/{id}`	OK	doc + attributes
`/documents` (collection) · `/users`	404	no list-all — search; users via Admin API

6 · THE 13 CURATED CLM TOOLS

Process: `list_workflow_definitions` · `launch_workflow` · `get_workflow_status`
Metadata: `get_attributes` · `set_attributes`
Find: `search_documents` · `list_folders` · `list_folder_documents`
Content: `get_document` · `download_document` · `upload_document` · `generate_document`
Ops: `account_info` (discovered hosts) · `raw_request` (everything else)

All prefixed `clm_`. Responses are trimmed before they reach the model; PDFs are written to disk and returned as a path rather than base64.

7 · WHAT CLM IS ACTUALLY FOR

Navigator holds the agreement *record*; CLM holds the *files, their metadata and the processes around them*. The demo-worthy answers live in two places:

- **Workflow definitions** — 87 here: contract intake routing, third-party paper with AI review and auto-redline from Ariba, amendment flows, clause capture, approval-before-signature. CLM as a process engine, not a file share.
- **Attribute groups** — *Procurement Routing Details* (Region, Category, Business Unit, Priority) and *CLM Agreement Details* (Agreement ID, Effective/Expiration Date). This is the structured contract data layer.

8 · FIELD NOTES THAT COST REAL TIME

Raw CLM paths start at the resource. `clmRequest()` already prepends `/{version}/{accountId}`. Passing `/v2/{accountId}/doclauncherconfigurations` double-prefixes and 404s — which reads exactly like "not entitled". It is `/doclauncherconfigurations`.

"CLM API requires a production account" appears on every CLM doc page. That is about *entitlement*, not environment — a CLM-provisioned UAT account works fine, as this one does. An early 401 here was a missing scope, nothing more.

Version is `v2`. Discovery does not return one. `v20180601`, `v201411` and `v20160301` all 404.

Name search truncates silently. `clm_search_documents` walks the folder tree under a visit cap; the repository is wide. Raise `max_folders` and surface `truncated: true` rather than reporting a partial result as complete.

9 · HOW THIS WAS BUILT

Specification-first would have failed: the published docs are wrong or ambiguous on all four points above. Every base URI, scope and path here was established by **probing the live account and recording what came back** — negative results included, since those are what save the next person a day. Contradictions with the docs are recorded as dated comments at the point of use in the source.

Built by Zach Woodward with AI pair-programming (Claude); correctness established by live-API verification, not by generation.